

Paradigmi di Programmazione

Enrico Melis

January 2026

Indice

1	Programmazione Orientata agli Oggetti	3
1.1	Le basi	3
1.2	Concetti avanzati	7
1.3	Garbage collection	10
2	Programmazione Concorrente	13
2.1	Concetti base	13
2.2	I problemi della concorrenza	13
2.3	Concorrenza in linguaggio high-level	14

1 Programmazione Orientata agli Oggetti

1.1 Le basi

La programmazione orientata agli oggetti (OOP) è un paradigma di programmazione che tenta di risolvere svariati problemi legati alla modularità del software, alla rappresentazione del mondo e ai vari livelli di astrazione. Alla base di questo paradigma c'è l'Oggetto, che non è altro che un'entità astratta dotata di stato e di funzionalità. Per chi ha familiarità col linguaggio C, possiamo dire che gli oggetti sono delle `struct` con la possibilità di aggiungere delle funzioni come campi.

1.1.1 Concetti principali

La OOP si basa sui seguenti concetti base:

1. Incapsulamento: occultamento dello stato interno dell'oggetto, visibile e modificabile soltanto tramite funzionalità definite all'interno del programma
2. Astrazione: qualsiasi oggetto modellabile tramite oggetto è anche astraibile a un livello superiore, definendo prima le caratteristiche più generali e piano piano specializzarle
3. Interfaccia: non è altro che la specifica astratta di un oggetto/classe
4. Ereditarietà: ogni oggetto di un tipo può avere un sottotipo, che estende le caratteristiche del supertipo ed è più specifico
5. Sostituzione: tramite le relazioni di sotto/super-tipo, possiamo definire dei comportamenti realizzabili da più tipi
6. Polimorfismo: possiamo definire comportamenti generici e renderli funzionanti per più tipi

1.1.2 Differenza fra classe e oggetto

I linguaggio orientati agli oggetti in realtà si dividono in due: *object-based* e *class-based*. Queste sono le differenze principali:

- Object: si può creare un oggetto in quanto entità a se stante, per i tipi si usa il *structural subtyping*. Esempi: OCaml, JavaScript.
- Class: si può creare un oggetto soltanto in forma di classe, per i tipi si usa il *nominal subtyping*. Esempi: Java, C++.

1.1.3 OOP in Java

Java è un linguaggio di programmazione *class-oriented*, nato nel 1995 e molto popolare al giorno d'oggi. È fortemente tipato e sfrutta un approccio misto di compilazione e interpretazione.

Il codice sorgente viene scritto in un file `File.java` che al suo interno contiene almeno una classe `File` che ha al suo interno un metodo statico `main()`. Il codice viene compilato (`javac File.java`) in un formato chiamato *bytecode* che viene successivamente interpretato dal JRE (Java Runtime Environment) tramite la JVM (Java Virtual Machine). Tutto questo permette a Java di essere un linguaggio "write once, execute anywhere" (a patto che ci sia una JVM).

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, _World!");
    }
}
```

Listing 1: Esempio di Hello World in Java

Interfacce Le interfacce in Java non sono altro che delle specifiche esplicite della struttura pubblica di una classe. Si può pensare come il prototipo di una classe che verrà implementata dopo.

```
public interface Esempio {
    public double metodo();
}

public class ImplementaInterfaccia implements Esempio {
    ...
    public double metodo() {
        System.out.println("Metodo_implementato");
    }
}
```

Listing 2: Esempio di Interfaccia

Una classe può implementare più interfacce alla volta.

Tipo apparente ed effettivo Quando parliamo di tipo apparente e tipo effettivo stiamo facendo riferimento a come cambia la tipatura di una variabile fra il tempo di compilazione e il tempo di esecuzione.

- Il tipo apparente è quello con cui viene dichiarata una variabile, è quello che usa il compilatore per fare i controlli statici
- Il tipo effettivo è quello a tempo di esecuzione, che potrebbe essere non noto a tempo di compilazione e che serve per risalire a una corretta chiamata di metodi

Ereditarietà Java supporta soltanto l'ereditarietà singola, non permette quella multipla fra classe come C++. Il concetto di ereditarietà introduce i sottotipi in Java, che come abbiamo detto in precedenza seguono il subtyping di tipo nominale. La keyword per l'ereditarietà in Java è **extends**.

```
public class Animale {
    protected String nome;

    public Animale(String nome) {
        this.nome = nome;
    }

    public void mangia() {
        System.out.println(nome + "_sta_mangiando");
    }
}
```

```
public class Cane extends Animale {

    public Cane(String nome) {
        super(nome);
    }

    public void abbaia() {
        System.out.println(nome + "_sta_abbaiano");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Cane fido = new Cane("Fido");
        fido.mangia();
    }
}
```

```
        fido.abbaia();  
    }  
}
```

Listing 3: Esempio di ereditarietà singola in Java

Dynamic dispatch Il dynamic dispatch è quel processo in cui si seleziona quale delle possibili implementazioni di un metodo polimorfo bisogna eseguire a runtime.

Nella Java Virtual Machine troviamo una soluzione efficiente che permette di non dover risalire ogni volta l'albero dell'ereditarietà. Funziona nel seguente modo:

- ogni classe ha una tabella di coppie (nome_metodo, locazione) e le sottoclassi hanno struttura inizialmente identiche (sharing strutturale)
- il compilatore determina staticamente l'offset del metodo da chiamare (dove si trova nella tabella)
- a runtime riesci a chiamare il metodo giusto perché: se il metodo non è *overrode*, allora ci sarà un puntatore al codice del metodo ereditato, altrimenti viene chiamata la nuova versione del metodo

Java Collection Framework Il JCF è una serie di implementazioni preconfezionate che Java ci offre per gestire in modo corretto e opinionato (ma comunque flessibile) i dati collezionabili in struttura dati classiche. Alcuni esempi sono:

- ArrayList
- Vector
- LinkedList
- HashSet
- HashTree
- LinkedHashSet
- e altri

1.2 Concetti avanzati

1.2.1 Ereditarietà multipla

Con ereditarietà multipla intendiamo la possibilità di dichiarare una classe che ne estende più di una contemporaneamente. In alcuni linguaggi è possibile farlo in modo nativo, in altri si possono usare altre modalità per ottenere risultato simili.

In C++ Il C++ supporta l'ereditarietà multipla nativamente.

```
#include <iostream>
#include <string>

class Animale {
protected:
    std::string nome;
public:
    Animale(const std::string& n) : nome(n) {}

    void respira() {
        std::cout << nome << "_sta_respirando."
        << std::endl;
    }
};

class Volante {
protected:
    int apertura_alare;
public:
    Volante(int apertura) : apertura_alare(apertura) {}

    void vola() {
        std::cout << "Sto_volando_con_apertura_alare_"
        << apertura_alare << "_cm."
        << std::endl;
    }
};

class Uccello : public Animale, public Volante {
public:
    Uccello(const std::string& n, int apertura)
```

```
        : Animale(n), Volante(apertura) {}

    void canta() {
        std::cout << nome << "_sta_cantando!"
        << std::endl;
    }

    void descrivi() {
        std::cout << "Sono_" << nome
        << ",_un_uccello_con_apertura_alare_"
        << apertura_alare
        << "_cm." << std::endl;
    }
};

int main() {
    Uccello passero("Passerotto", 25);

    passero.respira();
    passero.vola();
    passero.canta();
    passero.descrivi();

    return 0;
}
```

Listing 4: Esempio di ereditarietà multipla in C++

I problemi derivanti L'esempio è banale, ma è sufficiente per introdurre due problematiche standard dell'ereditarietà multipla: la risoluzione dei metodi e il "problema del diamante". Vediamo come vengono risolti in C++.

- Metodi: il dynamic dispatch dei metodi rimane molto simile; viene mantenuto lo sharing strutturale per la prima classe ereditata e le seconde hanno dei puntatori al codice dei metodi determinabile staticamente. Nel caso in cui una classe dovesse chiamare un metodo appartenente a più classi ereditate, si può fare disambiguazione esplicita (`c.A::foo()`; per chiamare il metodo giusto, supponendo che la classe `C` erediti sia da `A` che da `B`).
- Diamond: per risolvere il problema del diamante il C++ offre una soluzione semplice, che consiste nella keyword `virtual` al momento

della dichiarazione che segnala al compilatore che non deve costruire nuovamente una classe già ereditata nel caso potesse farlo. Esempio:

```
class A : virtual public B .
```

I mixins I mixins sono una soluzione al problema dell'ereditarietà multipla, non sono altro che una serie di metodi definiti in modo astratto a cui le classi possono fare riferimento durante la dichiarazione.

1.2.2 Principio di Sostituzione di Liskov

(*in inglese: LSP, Liskov's Substitution Principle* ¹)

Il principio di sostituzione di Liskov dice che un oggetto di una superclasse deve essere sostituibile con un oggetto della sottoclasse senza modificarne il comportamento; pertanto, i sottotipi devono mantenere retrocompatibilità con i comportamenti dei supertipi.

Questo ci serve perché noi sappiamo che possiamo sempre usare una sottoclasse al posto di una superclasse, grazie al subtyping, ma non è detto che questo abbia senso dal punto di vista dello scopo del programma. Il PSL ci garantisce coerenza sotto questo punto di vista.

Buttiamo giù (pari pari dalle slides) le fasi della scrittura di un programma

1. Gerarchia di classi e di interfacce
2. Membri pubblici di ogni classe
3. Specifiche esplicite di ogni membro pubblico (condizioni, comportamento)
4. Implementazione di test
5. Membri privati secondo incapsulamento
6. Codice delle classi, verificabile coi test

Regole Dal PSL possiamo derivare questo piccolo gruppo di regole:

- della Segnatura: gli oggetti del sottotipo devono avere tutti i metodi del supertipo; le signature del sottotipo devono essere compatibili con quelle del supertipo

¹Barbara Liskov, 1939, è un'informatica americana.

- dei Metodi: le chiamate dei metodi del sottotipo devono comportarsi come le chiamate del supertipo
- della Proprietà: il sottotipo deve preservare tutte le proprietà verificabili del supertipo

1.3 Garbage collection

1.3.1 Perché la GC

La Garbage Collection è una tecnica di gestione automatica della memoria dinamica di un processo in esecuzione, consiste nell'analizzare a runtime quelle che sono le zone della memoria che non sono più accessibili e de-allocarle per rimuovere i *memory leak*. Altri problemi risolti dalla garbage collection sono quelli dei *dangling pointers* (puntatori che fanno riferimento a zone di memoria già de-allocate, causati dagli alias) e della doppia de-allocazione (cercare di liberare una zona di memoria già libera).

La memoria dinamica è rappresentabile come un grafo e così facendo possiamo introdurre i concetti di **raggiungibilità** e **root set**.

Frammentazione Un problema noto nel contesto della gestione della memoria dinamica è quello della frammentazione, che si divide in:

- Interna: spreco di memoria all'interno di un blocco allocato (ad esempio: alloco 12kb per un processo da 10kb)
- Esterna: lo spazio libero in memoria è abbastanza per essere allocato ma non è contiguo, è sparso per la memoria

I blocchi della memoria dinamica possono essere di dimensione fissa o variabile.

1.3.2 Metodologie principali

La GC può essere realizzata tramite due metodologie principali, che sono il *reference counting* e il *tracing*.

Il primo funziona tramite un contatore che tiene traccia del numero di riferimento che puntano a una certa zona di memoria allocata, quando il numero di riferimenti va a 0, viene de-allocata. I problemi principali di questo approccio sono la costosità del controllo a tempo di esecuzione e i riferimenti circolati (se due oggetti si puntano a vicenda ma nessuno punta a loro, i counter non sono mai nulli ma nessuno può arrivare a quelle zone di memoria).

Il secondo funziona tramite il concetto di **raggiungibilità**: immaginando l'heap come un grafo, esiste una visita che mi permette di arrivare alla zona di memoria X? Abbiamo visto tre approcci al tracing, che sono:

- **Mark & Sweep**: ogni tot viene fatta una visita del grafo e i nodi raggiungibili sono marcati, tutte le zone di memoria non marcate vengono liberate
- **Copy Collection**: viene utilizzato metà memoria alla volta, ogni tot l'esecuzione viene fermata e si attraversa il grafo, copiando nell'altra metà tutte le regioni di memoria raggiungibili mentre quelle non raggiungibili vengono liberate
- **Generazionale**: si crea una sorta di "gerarchia" di oggetti in base a quella che è l'aspettativa di vita. Gli oggetti che hanno un'aspettativa di vita bassa vengono controllati più spesso rispetto a quelli con un'aspettativa di vita maggiore.

1.3.3 Rust

Rust è un linguaggio con un nucleo funzionale ed expression-first, nonostante sia multi-paradigma (procedurale, funzionale, oo). Ha un approccio alla gestione della memoria dinamica che ha come obiettivo la velocità del C e la sicurezza dei linguaggi meno permissivi; a discapito del programmatore che viene "flagellato" dalle regole imposte dal compilatore.

Rust introduce i concetti di *ownership*, *lifetime* e *borrowing* per la gestione della memoria dinamica. Questi servono ad assicurare che in qualsiasi momento ogni puntatore faccia riferimento a una zona di memoria valida, tutto controllabile staticamente a tempo di compilazione.

Ownership Ogni variabile ha al massimo un proprietario, non esistono alias, quindi non è possibile avere due puntatori per la stessa locazione di memoria. Al momento di un assegnamento, la proprietà del valore contenuto in una variabile viene passato alla variabile nuova, droppando quella iniziale (`drop()` in Rust $\simeq$ `free()` in C).

Lifetime Per lifetime di una variabile si intende il periodo in cui il suo valore è accessibile da qualcuno, quindi da quando viene creata a quando viene distrutta.

Borrowing La proprietà di una variabile può essere cambiata temporaneamente (prestata) tramite l'operatore unario `&`, che permette il passaggio di riferimenti, ad esempio a delle funzioni, permettendo però al chiamante di riacquisire il riferimento dopo il termine della funzione (altrimenti il riferimento verrebbe distrutto insieme alla funzione, senza ridare niente al chiamante).

```
fn main() {
    let s1 = String::from(" ciao ");
    let s2 = s1;
    println!("{}", s2);

    let data = vec![1, 2, 3];
    stampa(&data);
    stampa(&data);
    println!("{:?}", data);

    let mut numeri = vec![1, 2, 3];
    raddoppia(&mut numeri);
    println!("{:?}", numeri);

    let r;
    let y = 10;
    r = &y;
    println!("{}", r);
}

fn stampa(v: &Vec<i32>) {
    println!("{:?}", v);
}

fn raddoppia(v: &mut Vec<i32>) {
    for n in v.iter_mut() {
        *n *= 2;
    }
}

fn piu_lunga<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
```

Listing 5: Esempio di codice Rust

2 Programmazione Concorrente

2.1 Concetti base

Per programmazione concorrente si intende la possibilità di avere più processi che lavorano insieme. Astraiamo questo concetto dicendo che lavorano contemporaneamente, ma *concorrenza* e *parallelismo* sono due concetti diversi. Ogni sottoprocesso è un programma eseguito sequenzialmente che può cominciare con altri processi tramite *shared memory* o *message passing*.

2.1.1 Concorrenza vs. Parallelismo

Per concorrenza intendiamo l'esecuzione di N tasks su 1 unità di calcolo.
Per parallelismo intendiamo l'esecuzione di N tasks su N unità di calcolo.

2.1.2 Thread vs. Processi

Il processo può contenere più thread.

Un processo è un programma in esecuzione, con un suo ID, una sua regione di memoria allocata (dal sistema operativo).

Un thread è un'unità di esecuzione più piccola, all'interno di un processo, senza un ID e che condivide la memoria del processo con altri thread.

Esempio: le schede di Chrome. Il browser è un processo, ogni scheda è un thread.

2.1.3 Comunicazione interprocesso

La comunicazione fra processi (e/o fra thread) può avvenire tramite:

- Shared memory: p_1 e p_2 possono accedere alla stessa area di memoria condivisa e comunicano tramite variabili condivise
- Message passing: p_1 e p_2 usano dei costrutti IPC messi a disposizione dal sistema operativo per comunicare fra di loro

2.2 I problemi della concorrenza

2.2.1 Quando manca sincronizzazione

Avendo definito una sintassi che permette la concorrenza e delle regole (corrette) di inferenza, possiamo scatenare una serie di problematiche derivanti dal fatto che i thread/processi non sono coordinati fra di loro. Se due thread devono leggere la stessa variabile e incrementarla di 1, può accadere che il

primo legge e incrementa prima del secondo (risultato 2), ma se un legge subito dopo la lettura dell'altro avremo come risultato 1 perché entrambi hanno incrementato a partire dallo stesso valore.

2.2.2 Lock, Unlock e Mutex

Il **lock** è il blocco fondamentale per la costruzione di un meccanismo di mutua esclusione (*mutex*), così che un solo thread alla volta possa accedere a una risorsa condivisa. Introduzione l'idea di variabile m_n , cioè l' n -esimo mutex, che viene richiesto/bloccato tramite `lock  $m_n$` e rilasciato/sbloccato tramite `unlock  $m_n$` . Facendo così il thread blocca l'accesso a una certa area di memoria a cui fa riferimento il mutex.

[...]-grained locking I mutex possono fare riferimento ad aree di memoria più o meno grandi, questi sono i due casi estremi:

- *coarse-grained locking*: un unico mutex per una grande area di memoria, diminuisce l'effettiva concorrenza
- *fine-grained locking*: tanti mutex per tante piccole aree di memoria

2.2.3 Deadlock e soluzioni

Il deadlock è quello stato in cui due o più processo si bloccano a vicenda a causa di una richiesta non coordinata degli stessi mutex, rendendo impossibile l'avanzamento per tutti i richiedenti. Si risolve con:

- *prevention*: si permette la richiesta dei mutex soltanto in un modo specifico tramite regole di sintassi. esempio: 2-phase locking, in cui si mutex vengono richiesti in ordine crescente e rilasciati in ordine decrescente.
- *avoidance*: a runtime ci si accorge di situazioni che portano a deadlock e si interviene per cambiare l'ordine di esecuzione ed evitarlo
- *recovery*: si lascia andare il programma in deadlock ma comunque il runtime cerca successivamente di ripristinare l'esecuzione a uno stato senza deadlock

2.3 Concorrenza in linguaggio high-level

Dato che non ho voglia di scrivere più, dirò semplicemente che in Java esiste una classe `Thread` che all'interno ha un metodo `run()` che viene chiamato automaticamente dopo aver fatto

```
class X extends Thread {  
    public run(){}  
}  
...  
X t = new X();  
t.start();
```

Listing 6: Esempio di Thread in Java

Per avere una sincronizzazione migliore si usa la keyword **synchronized** [this].